

The Collections Framework

Package java.util

Strutture dati in java

- Molto spesso, una computazione si basa su una o più strutture dati, di vario tipo:
 - insiemi, code, stack, tabelle, liste, alberi...
- Data la loro importanza, Java ne offre un'ampia scelta nella *Java Collection Framework (JCF)*
 - interfacce che definiscono *insiemi di funzionalità*
 - classi che forniscono *varie implementazioni*
- Sono *contenitori "generici" per oggetti*
 - fino a JDK 1.4, si usava `Object` come tipo generico
 - da Java 1.5 in poi: supporto ai TIPI GENERICI

Cos'è una collezione (Collection)?

- Un oggetto che raggruppa un gruppo di elementi in una singolo oggetto.
- Uso: memorizzare, manipolare e trasmettere dati da un metodo ad un altro.
- Tipicamente rappresentano gruppi di elementi naturalmente collegati:
 - Una collezione di lettere
 - Una collezione di numeri di telefono

Cos'è l'ambiente “Collections”?

- E' un'architettura unificata per manipolare collezioni.
- Un framework di collection contiene tre elementi:
 - Interfacce
 - Implementazioni
 - Algoritmi
- Esempi
 - C++ Standard Template Library (STL)
 - Smalltalk's collection classes.
 - Java's collection framework

Cos'è l'ambiente “Collections”?

- ▣ Vantaggi
 - ▣ Riduce lo sforzo di programmazione
 - ▣ Accresce la velocità e la qualità di programmazione
 - ▣ Interoperabilità fra API scorrelate
 - ▣ Riduce lo sforzo di imparare ed utilizzare nuove API
 - ▣ Riduce lo sforzo per progettare nuove API
 - ▣ Riutilizzo del software

Collections Framework

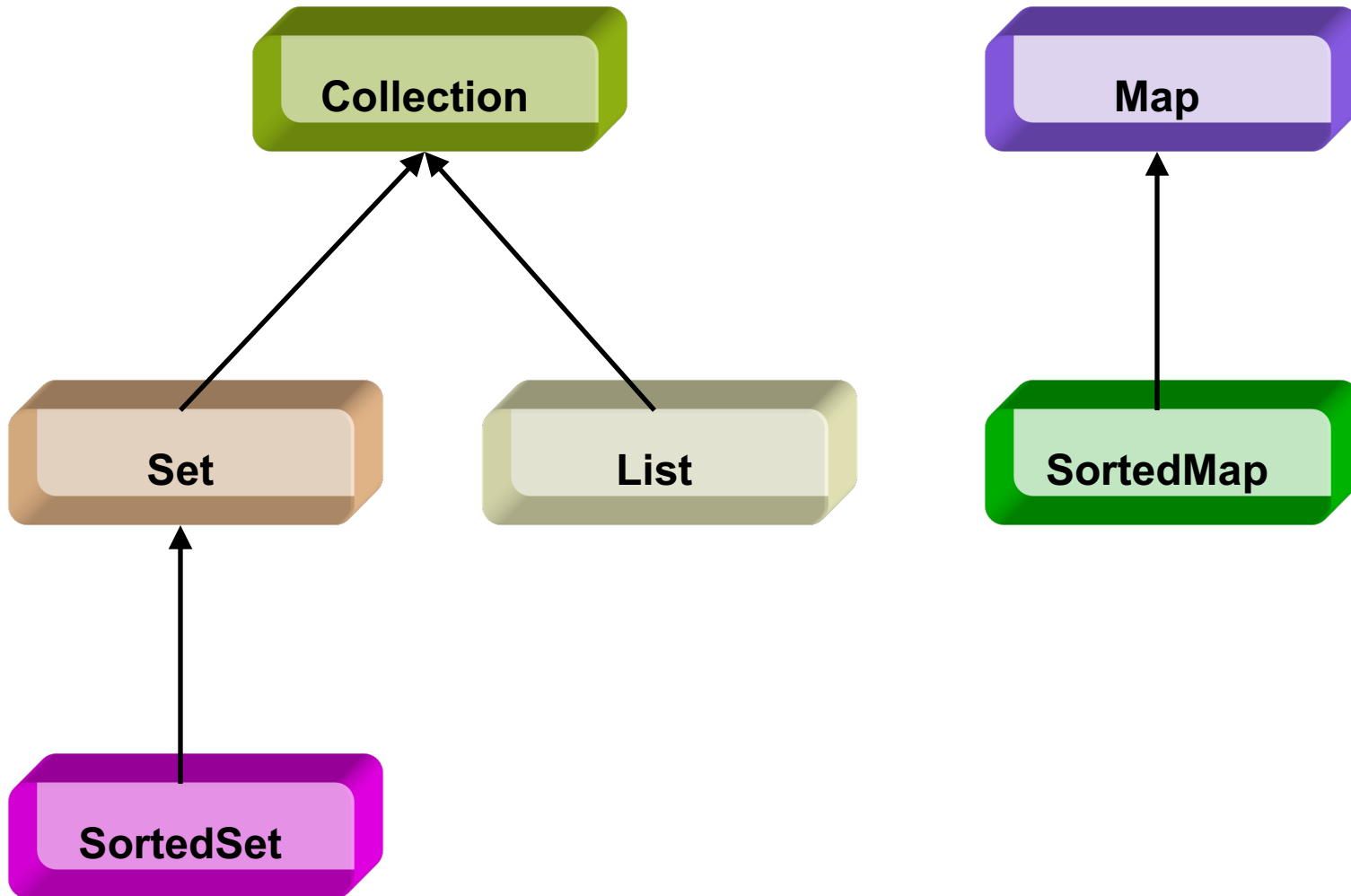
■ Interfacce

- Abstract data types rappresentanti le collezioni.
- Permettono di manipolare le collezioni indipendentemente dai dettagli delle loro implementazioni.
- In un linguaggio object-oriented come Java, queste interfacce formano una gerarchia

■ Algoritmi

- Metodi che eseguono utili computazioni come ricerca, ordinamento sugli oggetti che implementano le interfacce
- Questi algoritmi sono polimorfi in quanto lo stesso metodo può essere utilizzato su molte differenti implementazioni della stessa interfaccia.

Interfacce di Collections



JCF: Quadro generale

Implementazioni fondamentali

- ▣ per le liste: *ArrayList*, *LinkedList*, *Vector*
- ▣ per le tabelle hash: *HashMap*, *HashSet*, *Hashtable*
- ▣ per gli alberi: *TreeSet*, *TreeMap* (implementano rispettivamente *SortedSet* e *SortedMap*)

Due parole sulle classi di implementazione:

- ▣ le *linked list* sono liste realizzate con puntatori
- ▣ i *resizable array* sono liste di array
- ▣ i *balanced tree* sono alberi realizzati con puntatori
- ▣ le *tabelle hash* sono tabelle realizzate tramite *funzioni hash*, ossia funzioni che associano una entità a un valore tramite una funzione matematica.

JCF "classica"

Contenitori generici per oggetti realizzati tramite il tipo generico **Object**

JCF "CLASSICA" (fino a Java 1.4)

- Scelta di fondo: uso del tipo generico Object come mezzo per ottenere contenitori generici
 - i metodi che aggiungono / tolgono *oggetti* dalle collezioni prevedono un parametro di tipo Object
 - i metodi che cercano / restituiscono *oggetti* dalle collezioni prevedono un valore di ritorno Object
- Conseguenza:
 - si possono aggiungere/togliere alle/dalle collezioni oggetti di qualunque tipo, *TRANNE i tipi primitivi*
 - questi ultimi devono prima essere rispettivamente *incapsulati in un oggetto (BOXING)* / *estratti da un oggetto che li racchiuda (UNBOXING)*

Generics

Contenitori generici per oggetti realizzati
tramite il concetto di *tipo parametrico*

Il nuovo approccio (java 1.5)

- È sbagliato abolire il controllo di tipo!
 - Occorre *un altro modo* per esprimere genericità, che consenta un controllo di tipo a compile time
- type safety: *"se si compila, è certamente corretto"*
- Java 1.5 introduce i tipi parametrici ("generici")
 - Il tipo può essere un parametro:
 - in funzioni statiche, in classi e metodi di classi
 - notazione <TIPO>
 - Si possono definire relazioni fra "tipi generici". Recupera il "lato buono" dell'ereditarietà fra collezioni, inquadrandolo in un contesto solido.

Generics

- Un generic è un metodo che è ricompilato con differenti tipi secondo le necessità (simile ai template C++)
- Svantaggi:
 - Invece di : `ArrayList words = new ArrayList();`
 - Si deve definire:
`ArrayList<String> words = new ArrayList<String>();`
- Vantaggi:
 - Fornisce una migliore gestione del type checking durante la compilazione
 - Evita il casting da Object. i.e., invece di
`String title = ((String) words.get(i)).toUpperCase();`
utilizzeremo
`String title = words.get(i).toUpperCase();`

Esempio di classe parametrica

```
class GenericClass <T> {  
    T attributo;  
    public GenericClass (T x){  
        attributo = x;  
    }  
    public T getValue() {  
        return attributo;  
    }  
}
```

Uso della classe parametrica

```
public class Prova {  
    public static void main(String []s){  
        GenericClass <String> p1 = new GenericClass <String>(s[0]);  
        GenericClass <Integer> p2 = new GenericClass <Integer>(10);  
        String a = p1.getValue();  
        System.out.println(a);  
        Integer b = p2.getValue();  
        System.out.println(b);  
    }  
}
```

- Tutta la JFC è stata riscritta per far uso dei generici
 - Anche le classi preesistenti (come Vector) sono state reingegnerizzate e riscritte in accordo al nuovo idioma
 - Le operazioni sulla JFC "generica" si dicono *checked* (controllate) o *type-safe* (sicure come tipo)


```
List<String> ls = new ArrayList<String>(); //1  
List<Object> lo = ls; //2
```

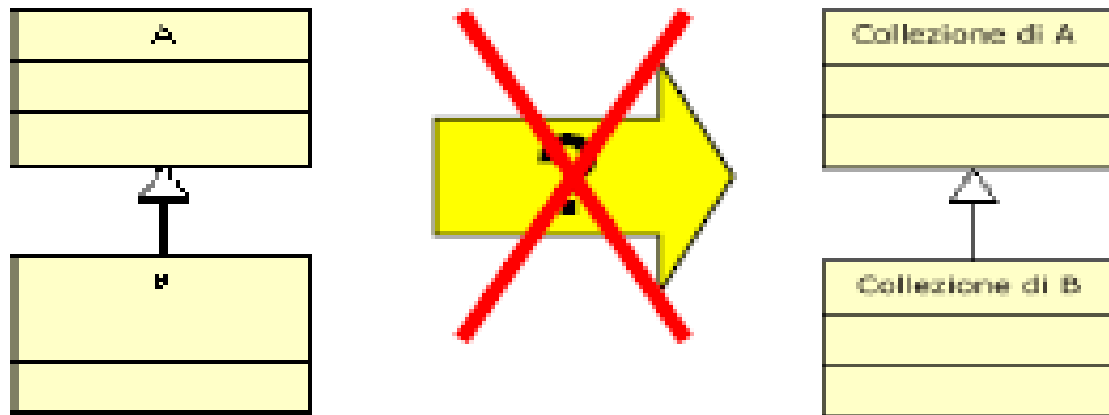
L'istruzione 2 genera un errore in compilazione

In generale, se C2 è una classe derivata da C1 e G è una dichiarazione generica, non è vero che G<C2> è un tipo derivato da G<C1>.

Tipi generici ed Ereditarietà

Verso il concetto di "tipo parametrico variante"

- Abbiamo visto che se B deriva da A, NON si può dire che una collezione di elementi di B derivi dalla collezione di elementi di A, perché in generale ciò non ha senso (operazioni impossibili)



ALCUNE operazioni potrebbero anche essere sicure (negli array, la lettura), ma ciò non è vero in generale.

Classi generiche ed ereditarietà

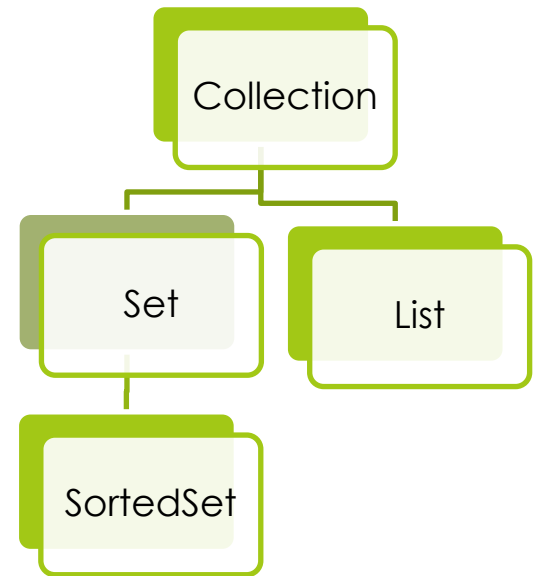
- Consideriamo la classe generica `LinkedList <T>`:
- prendiamo *due sue "istanziazioni"*
 - `LinkedList<Number>`
 - `LinkedList<Integer>`
- *Sono due tipi diversi, incompatibili fra loro!*
- Si evita così il rischio di situazioni "apparentemente giuste ma in realtà sbagliate" stile array.
- Per verificarlo, immaginiamo di creare due liste:
 - `LinkedList<Number> l1 = new LinkedList<Number>();`
 - `LinkedList<Integer> l2 = new LinkedList<Integer>();`
- e consideriamo i due possibili assegnamenti:
 - `l1 = l2` *//errore*
 - `l2 = l1` *//errore*

Collection

- Radice della gerarchia delle collezioni
- Rappresenta un gruppo di oggetti detti elementi della collezione.
- Alcune implementazioni di `Collection` consentono la duplicazione degli elementi mentre altre no. Alcune sono ordinate altre no.
- `Collection` è utilizzato per passare collezioni e manipolarle con la massima generalità.

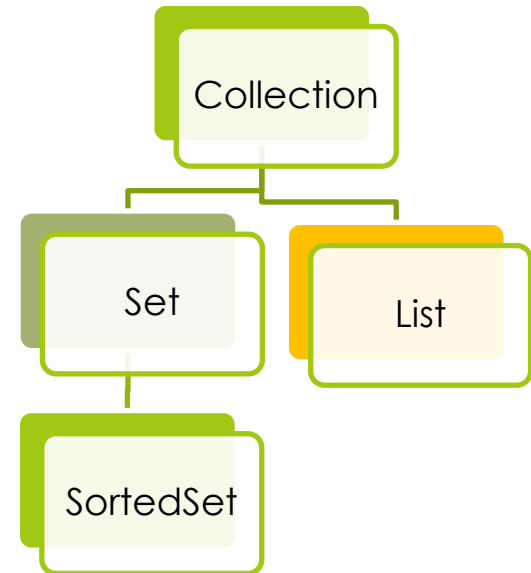
Set

- ❑ Interface `Set` extends `Collection`
- ❑ Modella l'astrazione matematica di insieme
 - ❑ Collezione non ordinata di object
- ❑ No elementi duplicati
- ❑ Gli stessi metodi di `Collection`
 - ❑ Semantica differente
- ❑ Implementazioni:
 - ❑ `AbstractSet`, `HashSet`, `LinkedHashSet`, `TreeSet`



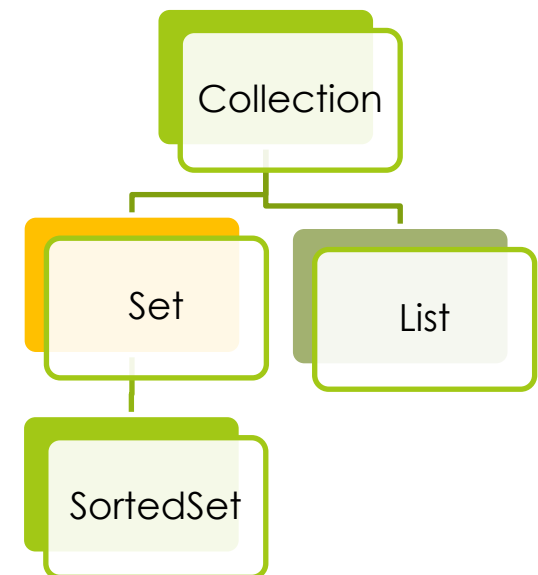
SortedSet

- ❑ Interface `SortedSet` extends `Set`
- ❑ Garantisce che l'insieme sia ordinato
- ❑ Tutti gli elementi inseriti in un sorted set devono implementare l'interfaccia `Comparable`
- ❑ **Implementazioni:**
 - ❑ `TreeSet`



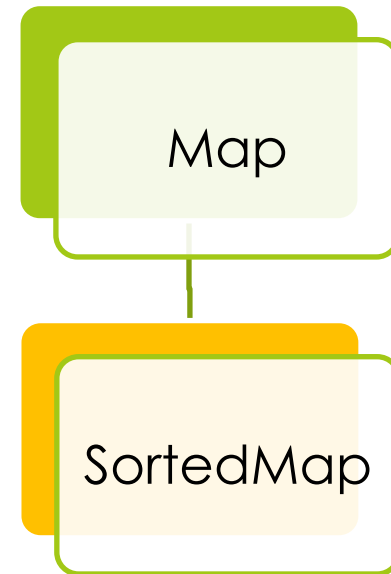
List

- Una collezione ordinata (chiamata anche sequenza).
- Può contenere elementi duplicati
- Consente il controllo della posizione nella lista in cui un elemento è inserito
- L'accesso agli elementi è eseguito rispetto al loro indice intero (posizione).
- **Implementazioni:**
 - AbstractList, ArrayList, LinkedList, Vector



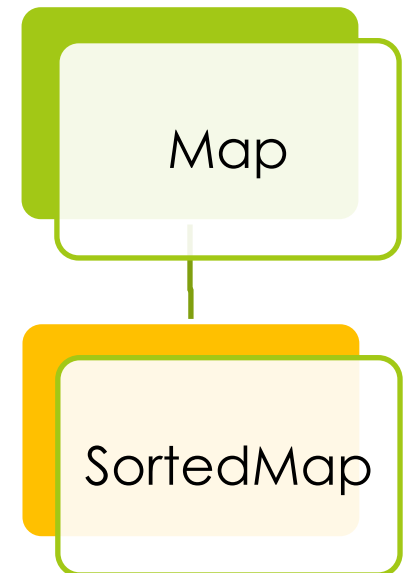
Map

- ❑ Interface Map (non estende Collection)
- ❑ Un object corrisponde (maps) valori chiave
- ❑ Non contiene chiavi duplicate
- ❑ Sostituisce la classe astratta `java.util.Dictionary`
- ❑ L'ordine può essere fornito da classi che implementano l'interfaccia
- ❑ **Implementazioni:**
 - ❑ [AbstractMap](#), [Attributes](#), [HashMap](#), [Hashtable](#),
[IdentityHashMap](#), [RenderingHints](#), [TreeMap](#), [WeakHashMap](#)



SortedMap

- public interface **SortedMap** extends Map
- una map che garantisce un ordinamento in senso crescente.
- Tutti gli elementi inseriti in un sorted map devono implementare l'interfaccia Comparable
- **Implementazioni:**
 - TreeMap



Implementazioni nel Framework Collections

- Le implementazioni concrete dell'interfaccia collection sono strutture dati riutilizzabili

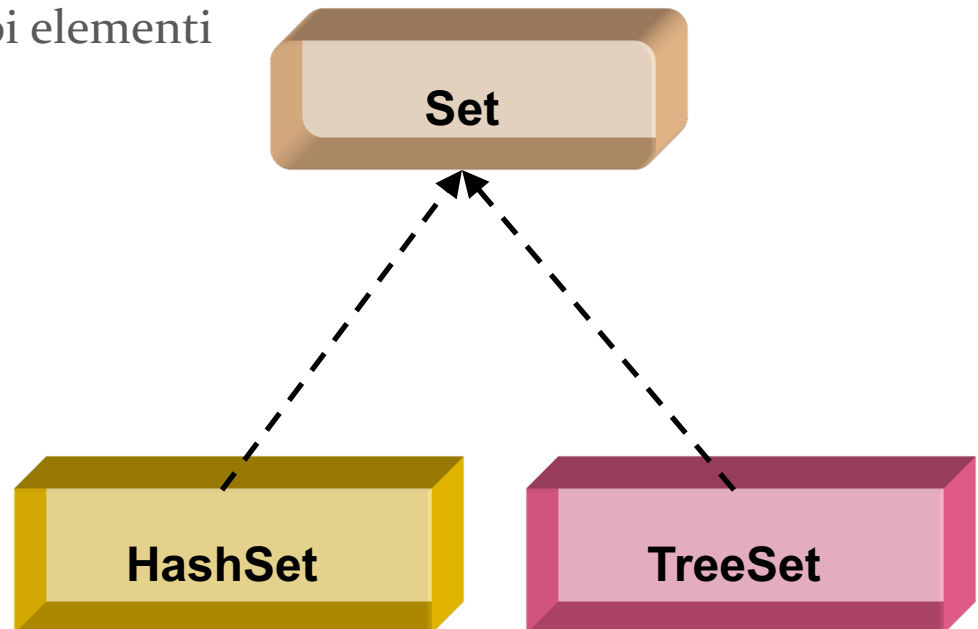
Set Implementations

- HashSet

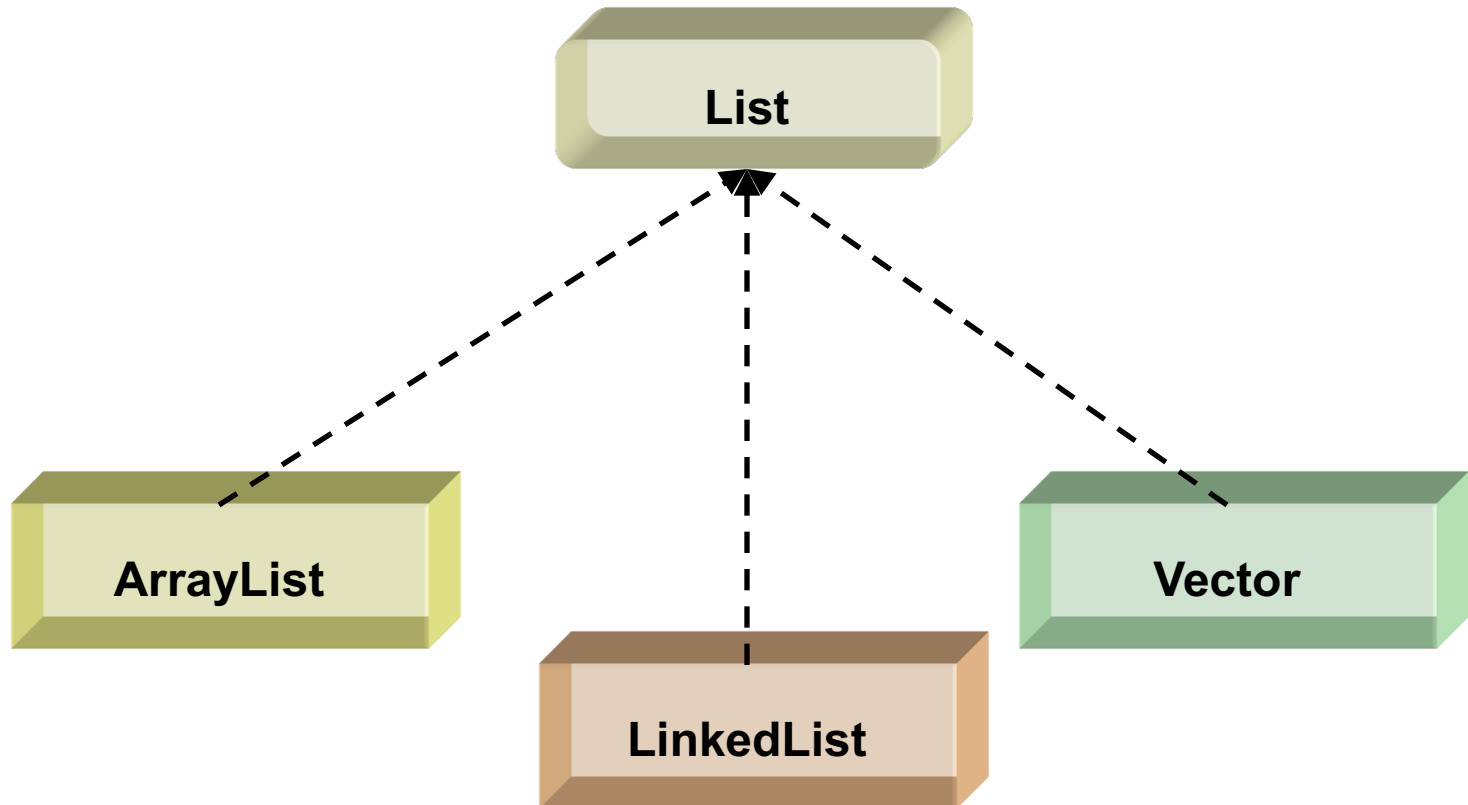
- un insieme associato con una hash table

- TreeSet

- Implementazione di un albero binario bilanciato
 - Impone un ordine nei suoi elementi



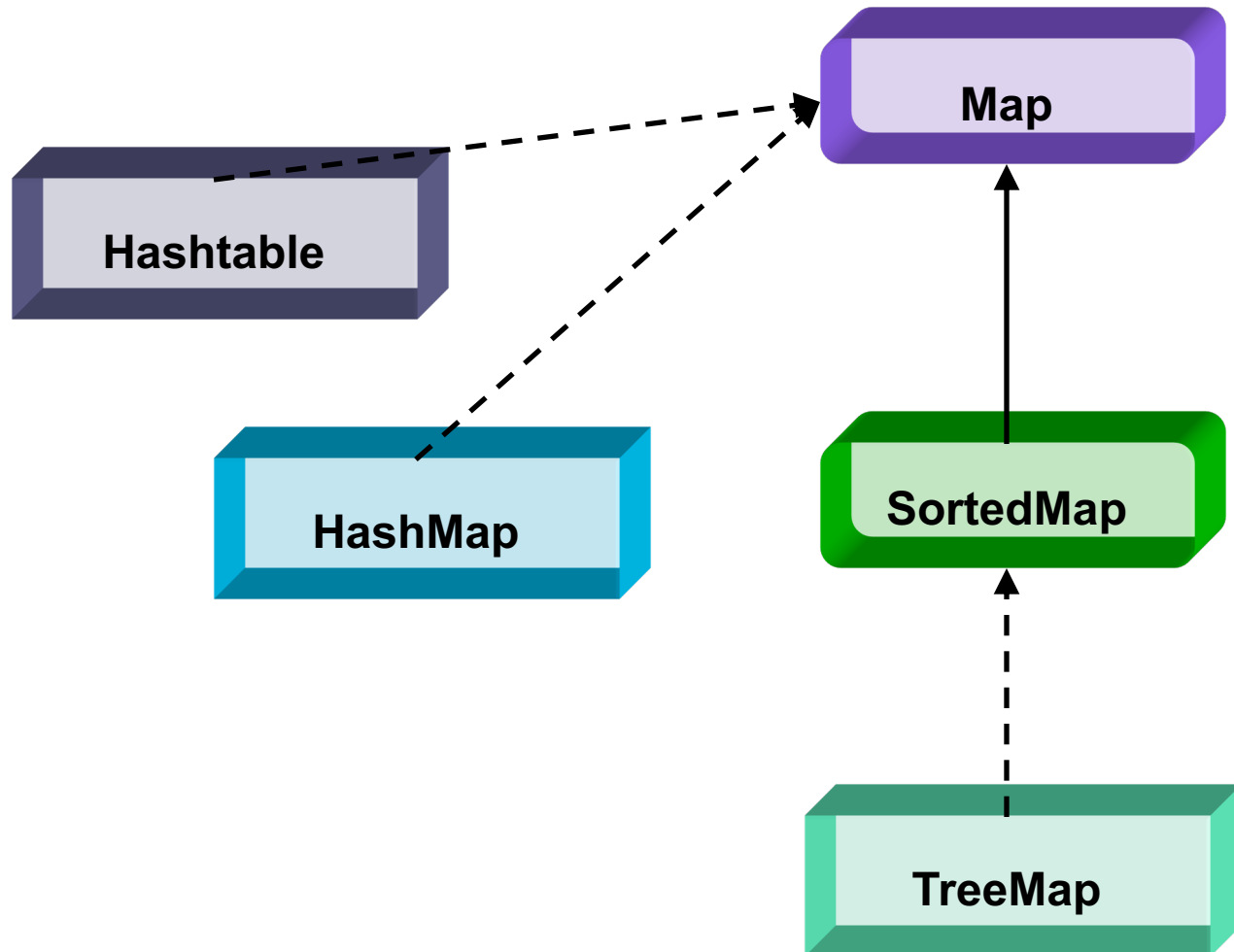
List Implementations



List Implementations

- ▣ ArrayList
 - ▣ Un array ridimensionabile
 - ▣ Asincrono
- ▣ LinkedList
 - ▣ Una lista doppiamente concatenata
 - ▣ Può avere performance migliori di ArrayList
- ▣ Vector
 - ▣ Un array ridimensionabile
 - ▣ Sincrono

Map Implementations

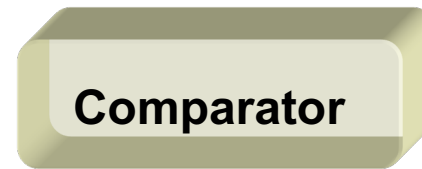
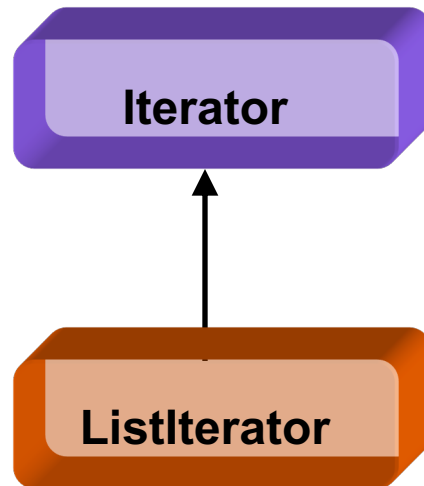


Map Implementations

- `HashMap`
 - Un hash table implementazione di `Map`
 - Come `Hashtable`, ma supporta null keys & values
 - Non sincronizzata
- `TreeMap`
 - Un albero binario bilanciato
 - Impone un ordine nelle chiavi
- `Hashtable`
 - hash table sincronizzato
 - Implementazione dell'interfaccia `Map`.

Utility di Collections Framework

Interfaces



Classes



Iterator

- public interface **Iterator**<E>
- Creato da `Collection.iterator()`
- Simile a `Enumeration`
 - Nome di metodi migliorati
 - Permette una operazione `remove()` sull'item corrente
- boolean **hasNext()**
Restituisce true se ci sono ancora elementi su cui iterare.
- **E next()**
Restituisce il successivo elemento.
- void **remove()**
rimuove l'elemento corrente

ListIterator

- public interface **ListIterator**<E> extends Iterator<E>
 - Attraversa la `List` in ogni direzione
 - Modifica `List` durante l'iterazione

Metodi aggiunti:

- void **add**(E e)
 - Inserisce un elemento nella lista.
- boolean **hasPrevious**()
 - Restituisce true se ci sono ancora elementi quando la lista si attraversa nella direzione inversa.

□ `int nextIndex()`

- Restituisce l'indice dell'elemento che dovrebbe essere restituito da una successiva chiamata al metodo `next`.

□ `E previous()`

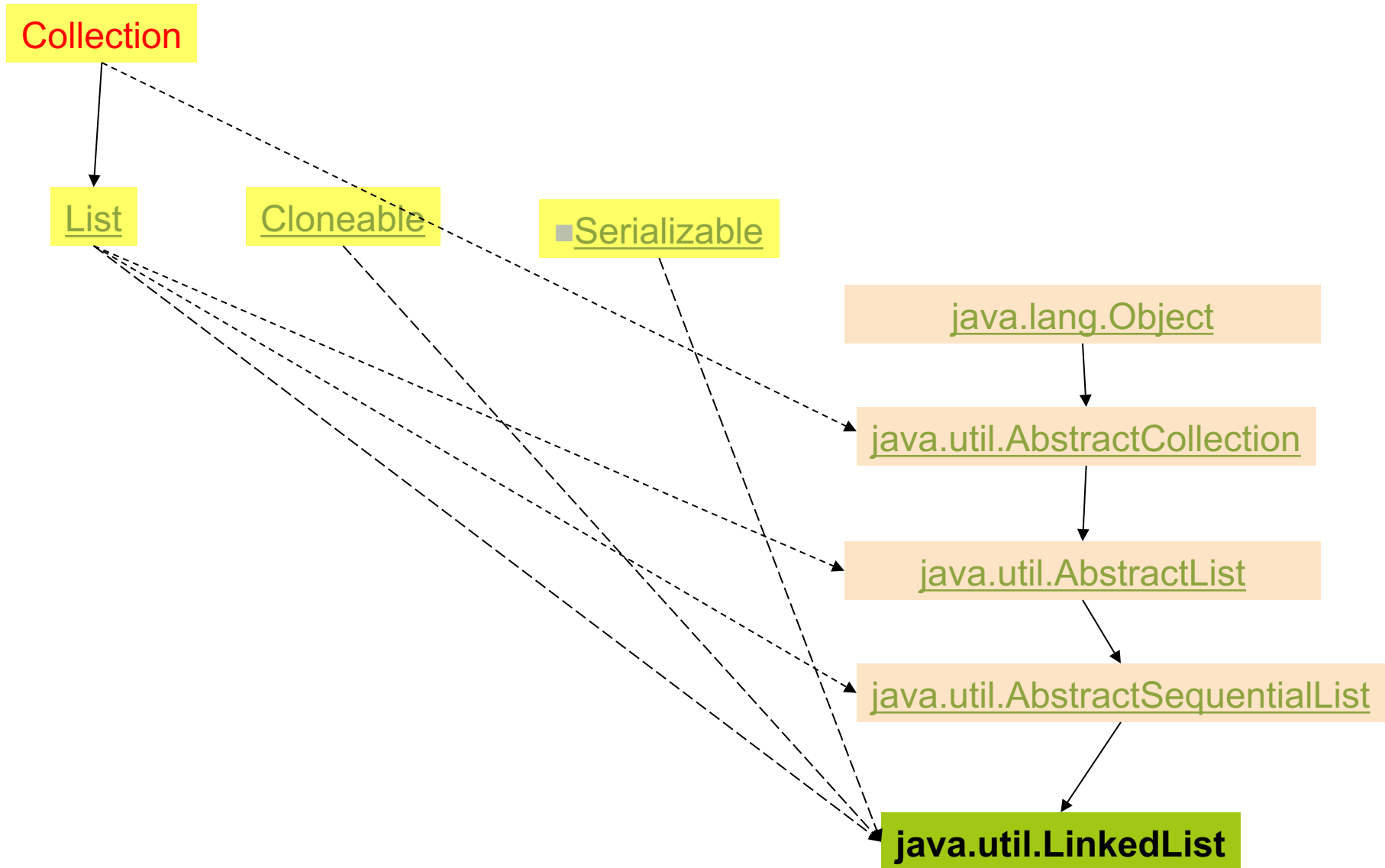
- Restituisce l'elemento precedente nella lista.

□ `int previousIndex()`

- Restituisce l'indice dell'elemento che dovrebbe essere restituito da una successiva chiamata al metodo `previous`.

□ `void set(E e)`

- Sostituisce l'elemento restituito da `next` o da `previous`.



Liste

- Nel package `java.util`
 - due tipi di liste
 - entrambe implementano l'interfaccia `java.util.List`
 - `ArrayList`: basata su array e indicatore di riempimento
 - `LinkedList`: basata su rappresentazione collegata
- Attenzione - i due tipi di liste hanno prestazioni diverse nelle operazioni fondamentali

- la rappresentazione con array facilita l'accesso agli elementi data la posizione ma penalizza gli inserimenti e le cancellazioni in mezzo alla lista
- è necessario spostare gli elementi su o giù

Viceversa

- LinkedList è più lenta nell'accesso agli elementi data la posizione
- accedere l'elemento in posizione i richiede la scansione di i riferimenti
- ma è più veloce negli inserimenti e nelle cancellazioni (approssimativamente costano quanto la scansione)

Un test di prestazioni

- Basato su circa 1000 operazioni
- tempi misurati in millisecondi

Type	Get	Insert	Remove
array	172	na	na
ArrayList	281	328	30484
LinkedList	5828	109	16
Vector	422	360	30781

fonte: Thinking in Java

Considerazioni sulle prestazioni

- gli array sono i più veloci, ma non consentono inserimenti e cancellazioni
- ArrayList è veloce nell'accesso agli elementi, lenta in inserimenti e cancellazioni in mezzo
- LinkedList è più lenta nell'accesso, ma decisamente più veloce in inserimenti e canc.
- Vector è più lenta di entrambe e non dovrebbe essere utilizzata
- Di conseguenza:
 - nel caso di liste a bassa dinamica, per ridurre i tempi di scansione è opportuno usare ArrayList
 - per liste ad alta dinamica, con frequenti inserimenti e cancellazioni conviene utilizzare LinkedList
- E che succede se devo cambiare tipo ? es: passare da ArrayList a LinkedList

Linea guida

- ❑ è opportuno programmare con le interfacce invece che con le implementazioni
- ❑ le interfacce riducono l'accoppiamento tra le classi e semplificano i cambiamenti
- ❑ Nel caso delle liste:
 - ❑ è opportuno utilizzarle per quanto possibile attraverso riferimenti di tipo `java.util.List`
- ❑ In questo modo
 - ❑ le modifiche sono semplificate
 - ❑ basta cambiare le poche istruzioni in cui gli oggetti di tipo lista sono creati cambiando la classe usata per l'implementazione
 - ❑ il resto dell'applicazione resta intatta
 - ❑ i metodi si comportano polimorficamente e viene utilizzata la nuova implementazione

Iteratori

- Attenzione

- in questo approccio, quando manipolo la lista devo tenere in considerazione che l'implementazione potrebbe cambiare

- In particolare

- devo fare attenzione a non basare la scrittura del codice su una o l'altra delle implementazioni

- Un'operazione critica: la scansione

- Il modo tipico di scandire una lista utilizzando indici interi

```
for (int i = 0; i < lista.size(); i++) {  
    Object o = lista.get(i);  
    // operazioni su o  
}
```

Iteratori

- Questo tipo di scansione
 - è particolarmente adatto ad ArrayList (il metodo get viene eseguito rapidamente)
 - ma disastrosamente lenta su LinkedList
- Perché ?
 - perchè come detto l'accesso all'elemento in posizione i di una LinkedList richiede di scandire i elementi (i operazioni circa)

Detta n la dimensione della lista $1 + 2 + 3 + 4 + \dots + n$

- pari circa a $n(n+1)/2$, ovvero dell'ordine di n^2
 - es: per una lista di 100 elementi: 5000
 - nel caso di ArrayList: circa 100 operazioni
- In casi normali
 - il problema non sorge (liste piccole)
 - ma in alcuni casi si tratta di un costo di calcolo che può diventare inaccettabile

Il problema

- la scansione attraverso indici interi NON è la scansione più naturale per LinkedList
- ArrayList
 - implementazione basata su indici >> scansione naturale basata su indici
- LinkedList
 - implementazione basata su riferimenti >> scansione naturale basata su riferimenti
- Idealmente
 - vorrei che per ciascuna tipologia di lista potesse essere utilizzata automaticamente la scansione più adatta
 - senza che il programmatore se ne debba preoccupare
- Attenzione
 - in questo caso il polimorfismo da solo non basta

- La scansione della lista è un'operazione che deve necessariamente essere effettuata da un oggetto diverso dalla lista, non posso quindi semplicemente sovrascrivere il metodo “scandisciti()” e utilizzarlo polimorficamente devo necessariamente definire altri oggetti la cui responsabilità è quella di scandire la lista
- Soluzione
 - utilizzare un oggetto “iteratore”
- Iteratore
 - oggetto specializzato nella scansione di una lista
 - fornisce al programmatore un'interfaccia per effettuare la scansione in modo indipendente dalla strategia di scansione concreta (indici, puntatori, ecc.)
 - implementa la scansione in modo ottimale per ciascun tipo di lista

L'utilizzo in java.util

- interfaccia java.util.Iterator, che prevede i seguenti metodi
 - Object next() per spostarsi in avanti
 - boolean hasNext() per fermarsi
- esiste una implementazione per ArrayList
- ed una implementazione per LinkedList
- Iteratore per ArrayList
 - utilizza indici interi
- Iteratore per LinkedList
 - Scandisce la lista usando i riferimenti
- Come si ottiene l'iteratore ?
 - utilizzando il metodo Iterator iterator() di java.util.List

Dettagli sugli iteratori di java.util

- sostanzialmente si basano sui metodi `next()` e `previous()` forniti dalle due liste
- sono però più complicati di quanto si pensa dal momento che consentono anche di modificare la lista durante la scansione attraverso il metodo `remove()` senza doversi preoccupare della consistenza dei riferimenti

Ricerca e rimozione di un elemento

- E' necessario che la classe degli elementi contenuti nella lista implementi il metodo equals

Mappe

- Una mappa, ovvero un dizionario associativo
 - classe `java.util.HashMap`
 - implementa l'interfaccia `java.util.Map`
- Dizionario associativo
 - collezione in cui i riferimenti sono salvati con un “nome”, detto chiave
 - tipicamente una stringa
 - possono successivamente essere recuperati rapidamente utilizzando la chiave

```
add(new Giorno(2004, 6, 14));
```

```
add(new Giorno(2004, 6, 15));
```

```
get(0)
```

```
14/6/2004
```

java.util.List

elemento 0: 14/6/2004

elemento 1: 15/6/2004

```
put("primo", new Giorno(2004, 6, 14));
```

```
put("secondo", new Giorno(2004, 6, 15));
```

```
get("secondo")
```

```
15/6/2004
```

java.util.Map

"primo": 14/6/2004

"secondo": 15/6/2004

public interface Map<K,V>

Principali metodi

V put(K key, V value)

void putAll(Map<? extends K,? extends V> m)

V get(Object key)

V remove(Object key)

Implementazioni

- ▣ java.util.HashMap
- ▣ java.util.TreeMap
- ▣ java.util.Hashtable

metodi principali di `java.util.Map`

- ❑ `void put(Object chiave, Object riferimento)`
- ❑ `Object get(Object chiave)`
- ❑ `Object remove(Object chiave)`
- ❑ `int size()`

Le principali implementazioni

- ❑ `java.util.HashMap`
- ❑ `java.util.TreeMap`
- ❑ `java.util.Hashtable` (versione legacy)

Differenze rispetto alle liste

- in una mappa non sono significative le posizioni degli elementi ma le chiavi
- le ricerche sulla base della chiave sono enormemente facilitate (nella lista richiederebbero una scansione)
- utilizzata tipicamente quando più che le scansioni sono importanti le ricerche

Attenzione però

- ad ogni chiave può essere associato un unico oggetto
- put successive con la stessa chiave sostituiscono i valori precedenti
- non può essere usata quando possono esserci più valori per la stessa chiave

HashMap

- Un requisito fondamentale per le mappe
 - la rapidità di inserimento e cancellazione

L'implementazione fondamentale

- HashMap
 - di gran lunga la più veloce

La tecnica sottostante

- tecnica di hashing ovvero basata su “funzioni di hashing”

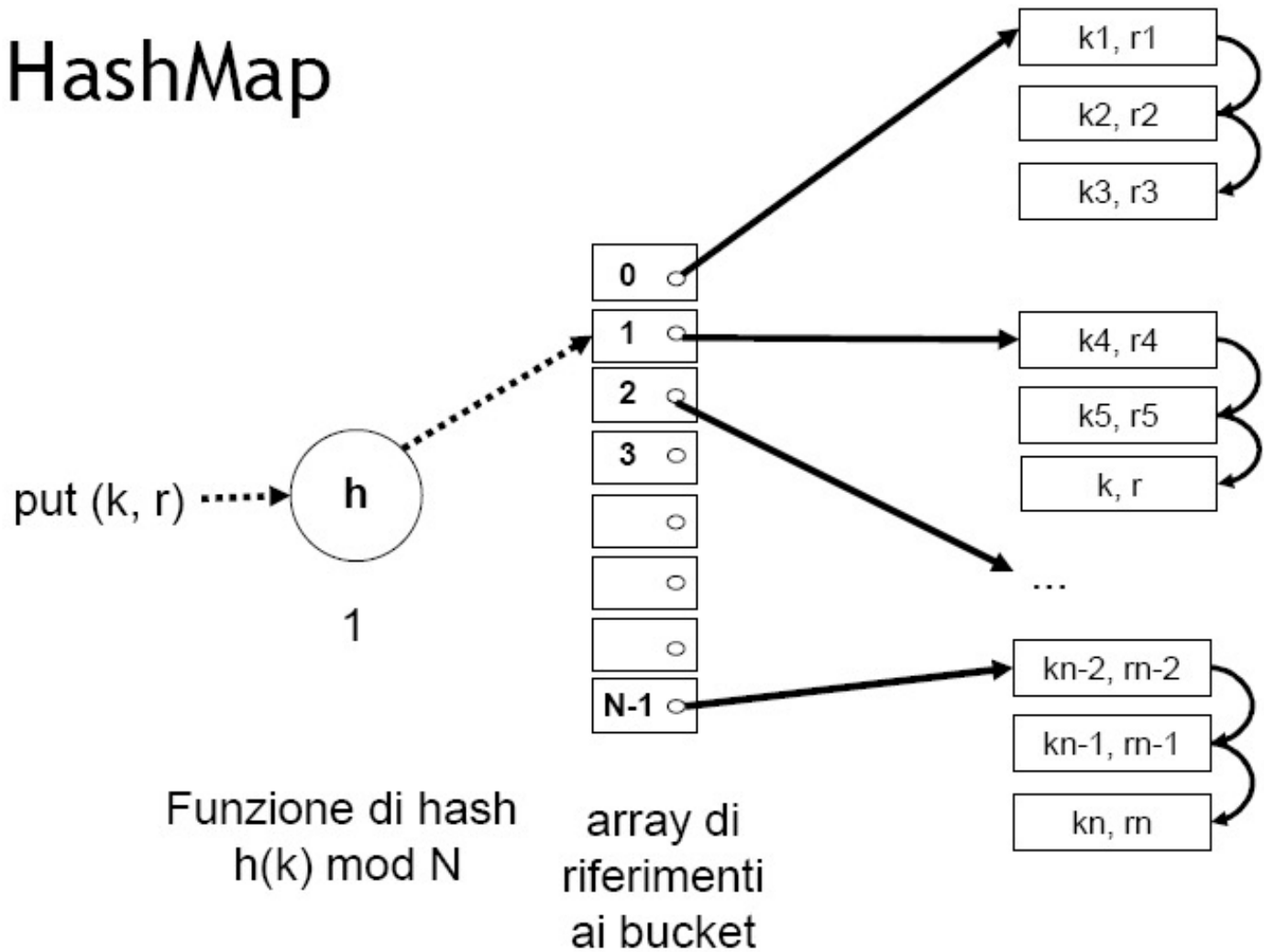
HashMap

- Funzione di hash
 - funzione che trasforma un valore (“chiave”) di lunghezza variabile in uno di lunghezza fissa (“hash” della chiave)
- Caratteristica tipica di una funzione hash
 - l’hash deve essere calcolabile rapidamente
- Classificazione delle funzioni hash
 - funzioni con collisioni o prive di collisioni
- Funzione priva di collisione
 - non ci sono due valori per cui l’hash è uguale
 - possibile solo se i valori sono finiti
- Funzione con collisione
 - più valori possono avere lo stesso valore di hash
 - caso tipico

HashMap

- Implementazione di `put()` nella HashMap
 - la mappa mantiene gli oggetti in un array di N riferimenti a liste (dette “bucket”)
 - ogni elemento della lista memorizza una coppia <chiave, riferimento>
 - viene calcolato il valore della funzione di hash sulla chiave e poi viene applicato un operatore modulo per ridurlo ad un numero tra 0 e $N - 1$
 - in questo modo si ottiene un indice nell'array; la coppia <chiave, riferimento> viene aggiunta in coda al bucket della posizione ottenuta

HashMap



HashMap

- Implementazione di `get()` in `HashMap`
 - viene calcolato il valore di hash della chiave per risalire al bucket (indice nell'array)
 - viene scandito il bucket e la chiave viene confrontata con ogni chiave
 - se viene trovata una chiave identica a quella cercata, viene restituito il riferimento
 - altrimenti viene restituito `null`
- Due operazioni fondamentali
 - il calcolo della funzione di hash
 - il confronto tra le chiavi
- Calcolo della funzione di hash
 - viene usato il metodo `hashCode()` ereditato da `Object`
- Confronto tra le chiavi
 - viene utilizzato il metodo `equals()` ereditato da `Object`

HashMap

- Nota

- le implementazioni standard sono basate sull'indirizzo in memoria
- potrebbero non essere quelle adatte a fare hashing in alcuni casi

- Nelle classi principali della piattaforma

- sono ridefinite opportunamente quando è necessario

- Di conseguenza

- è opportuno utilizzare come chiave per le mappe oggetti di classi note es: String, Integer, ecc.

- Nel caso in cui questo non sia possibile

- per la classe di oggetti da utilizzare come chiavi è necessario ridefinire opportunamente hashCode() ed equals()

```
public class HashMap<K,V> extends AbstractMap<K,V>  
    implements Map<K,V>, Cloneable, Serializable
```

- ▣ *Capacity*: numero di buckets nella hash table,
- ▣ *Load factor* e' la misura di quanto piena puo' essere l'hash table prima di un resize (0,75)
- ▣ Costruttori

HashMap() costruisce una HashMap vuota con capacity =16 e load factor = 0.75

HashMap(int initialCapacity) costruisce una HashMap vuota con capacity = initialCapacity e load factor = 0.75

HashMap(int initialCapacity, float loadFactor) costruisce HashMap con i valori specificati

HashMap(Map<? extends K,? extends V> m)
costruisce una HashMap con lo stesso mappings della mappa specificata.

Insiemi

- Altre classi significative delle collezioni
 - interface `java.util.Set`: rappresenta una collezione non ordinata e priva di duplicati
 - due principali implementazioni: `HashSet` e `TreeSet`
 - interface `java.util.Collection`: rappresenta una collezione generica (può essere un insieme oppure una lista)
 - tutte le collezioni sono scandibili con iteratori
- A cosa servono gli insiemi ?
 - possono essere quasi sempre sostituiti dalle liste
 - in alcuni casi sono più naturali
- Esempio: Gestione Appuntamenti
 - potresti utilizzare un oggetto di tipo `java.util.Set` per rappresentare l'insieme dei partecipanti ad una Riunione

Insiemi

- A questo punto possiamo vedere come scandire un Set
- Due metodi principali
 - ottengo l'insieme delle chiavi con Set `keySet()` e scandisco l'insieme con un iteratore e prelevo dalla mappa tutti gli elementi
 - ottengo la collezione dei valori con il metodo `Collection values()` ottengo un iteratore dalla collezione e scandisco tutti gli elementi
- Perché Set e Collection ?
 - per rispettare la regola secondo cui si programma con le interfacce e non con le implementazioni
- Infatti
 - Una `HashMap` è una collezione non ordinata cioè mantiene un insieme di chiavi e non una lista

Ordinamenti

- In Java

- L'ordinamento delle collezioni è una funzionalità offerta dalla piattaforma es: `java.util.Collections.sort (java.util.List list)` ma bisogna rispettare delle regole

- In particolare

- per ordinare una collezione di riferimenti, gli oggetti relativi devono essere confrontabili
- rispetto ad una precisa relazione di ordine

- Esempio

- la lista degli Impegni
- un ordinamento possibile: per orario
- un altro ordinamento: in ordine alfabetico rispetto alla descrizione
- un altro ordinamento ancora: rispetto al luogo di svolgimento
- per poter ordinare una lista di impegni bisogna decidere il criterio di ordinamento

- In Java

- gli oggetti ordinabili devono implementare l'interfaccia `java.lang.Comparable`

- `java.lang.Comparable`

- prevede un unico metodo
 - `int compareTo(Object o)`
 - risultato > 0 se `this` segue `o` nell'ordinamento
 - risultato < 0 se `this` precede `o` nell'ordinamento.
 - risultato $= 0$ se `this` non precede nè segue `o`

Ordinamenti

■ Nota

- l'utilizzo di Comparable prevede che sia possibile cambiare il codice delle classi che definiscono gli oggetti da ordinare
- per implementare Comparable e definire compareTo()
- se questo non è possibile, è possibile utilizzare un approccio alternativo
- utilizzare un oggetto esterno comparatore

■ L'interfaccia `java.util.Comparator`

- unico metodo: `int compare(Object o1, Object o2)`
- gli oggetti che implementano l'interfaccia sono quindi capaci di confrontare oggetti secondo algoritmi specifici